

PREMESSA: Su GitHub è possibile reperire i file necessari. Tutte le operazioni sotto descritte sono state automatizzate tramite uno script bash, *start.sh*.

DOWNLOAD DEI FILE NECESSARI

Secondo quanto riportata dalla guida ufficiale di OpenMetadata, <https://docs.open-metadata.org/v2.0.x/quick-start/local-docker-deployment#local-docker-deployment>, dobbiamo scaricare il file `docker-compose.yml`:

```
davide@Air-di-Davide PostgreSQL % mkdir openmetadata-docker && cd openmetadata-docker
davide@Air-di-Davide openmetadata-docker % curl -sL -o docker-compose.yml https://github.com/open-metadata/OpenMetadata/releases/download/2.0.1-release/docker-compose.yml
davide@Air-di-Davide openmetadata-docker % ls
docker-compose.yml
```

CREAZIONE DELLA RETE

Definiamo ora una rete tramite docker, la quale consentirà al container OpenMetadata e al container PostgreSQL di comunicare:

```
davide@Air-di-Davide openmetadata-docker % docker network create rete_pg
99bb17813c2b17fd2f194d1e29930ed03ca52e1396579f30215d9d1e4a8b9868
davide@Air-di-Davide openmetadata-docker % docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
6d84c0a10c93        bridge             bridge              local
3b2ece401f05        host               host                local
d822297f906b        none              null                local
99bb17813c2b        rete_pg            bridge              local
davide@Air-di-Davide openmetadata-docker %
```

MODIFICA DEL COMPOSE DI OPENMETADATA

Bisogna modificare il `docker-compose` scaricato in precedenza, in modo che sia specificata la rete sopra definita e che tutti i servizi la utilizzino. Il file modificato è reperibile su GitHub.

CREAZIONE DEL DOCKER-COMPOSE DI POSTGRES

Creiamo una directory *Postgres* in cui specifichiamo il `docker-compose` per creare il container ed uno script (*init.sql*) da eseguire per creare la tabella con i dati. Inoltre, specifichiamo la creazione di un utente OpenMetadata e gli diamo i permessi necessari. Anche questo file può essere reperito e visualizzato su GitHub.

ESECUZIONE

Avendo creato la rete possiamo eseguire il docker-compose di Postgres e quello di OpenMetadata.

```
davide@Air-di-Davide ~ % docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
8ccc213a63e9   docker.getcollate.io/openmetadata/server:2.0.0   "/bin/bash ./bootstr..." About a minute ago Up 31 seconds 8585-8586/tcp
d1a50b71ca7    docker.getcollate.io/openmetadata/db:2.0.0       "/entrypoint.sh --so..." About a minute ago Up About a minute (healthy) 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
openmetadata_mysql 4c54d8dd3a5d   docker.elastic.co/elasticsearch/elasticsearch:9.3.0 "/bin/tini -- /usr/L..." About a minute ago Up About a minute (healthy) 0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 0.0.0.0:9300->9300/tcp, [::]:9300->9300/tcp
openmetadata_elasticsearch bdfdd64283bd   postgres:17             "docker-entrypoint.s..." 7 minutes ago Up 7 minutes 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
pg-progetto
davide@Air-di-Davide ~ %
```

TEST DA TERMINALE

Per verificare che il tutto funzioni eseguiamo uno shell bash all'interno del container di openmetadata e testiamo la connessione al database Postgres:

```
davide@Air-di-Davide ~ % docker exec -it openmetadata_server bash
7bd49751d42d:/opt/openmetadata$ nc -zv pg-progetto 5432
pg-progetto (172.18.0.2:5432) open
7bd49751d42d:/opt/openmetadata$
```

TEST DA BROWSER

Infine, come ulteriore conferma, contattiamo il server OpenMetadata su <http://localhost:8585>. Inserire come credenziali quelle di default:

- Email: admin@open-metadata.org
- Password: admin

Una volta effettuato il login con successo selezionare Settings (dal menù laterale) -> Services -> Databases -> Add New Server -> Postgres.

Saranno richiesti dei campi di cui alcuni obbligatori:

- Service Name: progetto_postgres
- Username: openmetadata_user
- Host and Port: pg-progetto:5432

- Database: testmisurazioni
- Password (sezione 2, Authentication): openmetadata_pwd

Name this service
A unique name for this Postgres connection in OpenMetadata. You'll see it in the catalog and lineage.

Service Name *

progetto_postgres

+ Add a description

1 Connection Optional
Where OpenMetadata connects. These identify the warehouse and session.

Username *

openmetadata_user

Host and Port *

pg-progetto:5432

Database *

postgres

Username to connect to Postgres. This user should have privileges to read all the metadata in Postgres.

Host and port of the source service.

Initial database to connect to. Metadata reading is restricted to this database unless Ingest All Databases is enabled, in which case this database is used as the entry point to discover and scan all databases.

2 Authentication Optional
Choose one method. Exactly one credential is stored per connection — switching discards the other.

Auth Configuration Type

Basic Auth IAM Auth Configuration Source Azure Configuration Source

Password

Password to connect to source.

Requirements

Metadata

Note that we only support officially supported Postgres versions. You can check the version list [here](#).

Metadata Extraction

OpenMetadata extracts metadata from PostgreSQL using two main sources:

PostgreSQL System Catalogs (pg_*)

The connector primarily uses PostgreSQL's system catalog tables and views to extract metadata. These include:

- pg_class - Information about tables, views, and other relations
- pg_namespace - Schema information
- pg_attribute - Column information
- pg_description - Comments on database objects
- pg_constraint - Constraint information
- pg_database - Database information
- pg_tables - Table ownership and metadata
- pg_partitioned_table - Partitioning information
- pg_policy - Row-level security policies
- pg_sequence - Sequence information
- pg_attrdef - Column default values
- pg_proc - Stored procedures and functions

Premere sul pulsante *Test Connection* in fondo alla pagina:

Connection status
pg-progetto:5432

Test connection partially successful: Some steps had failures, we will only ingest partial metadata.

Establish Connection
Reached the service and opened a session. [Gate](#)

CAPABILITY CHECKS 8/9 passed

Check	Method	Status
List databases	GetDatabases	Passed
List schemas	GetSchemas	Passed
List tables	GetTables	Passed
List views	GetViews	Passed
Read tags / classifications	GetTags	Passed
Read query history (usage & lineage)	GetQueries	Warning
Get Column Metadata	GetColumnMetadata	Passed
Get Table Comments	GetTableComments	Passed
Get Information Schema Columns	GetInformationSchemaColumns	Passed

[Show raw connection log \(70\)](#)

[Copy log](#) [Done](#)

Nota: il warning apparso non è bloccante. Si tratta infatti di un'estensione che può essere installata a parte, la connessione è stata stabilita con successo.